

Fundamentals of AI and ML

Project Report

Analysis and Comparison of State-Space Search Algorithms

Submitted by: Arghya Avirbhav

Registration Number: 25BCE10859

Branch: B.Tech CSE

University: VIT Bhopal

Course Code: CSA2001

This project explores the fundamental "search" aspect of Artificial Intelligence, and analyses and compares two of the methods used for the same. By implementing Breadth First Search (BFS) and A* Search, the problem and its application demonstrate the transition from uninformed search (blindly going through every possibility) to informed search (using specific knowledge to search effectively). The project also gives a showcase of methods to evaluate how heuristics improve computational efficiency in pathfinding search tasks.

Problem Statement

The objective of both searches involved in this project is to find the most efficient path between a certain starting state (Node A) and a goal state (Node F) within a weighted and specified graph. The challenge is to compare Breadth First Search and A* Search.

We define the search methods involved as follows:

BFS: To find the shortest path in terms of the number of edges.

A* Search: To find the path with the minimum cumulative weight using a heuristic function.

Theoretical Framework

Using our implementation of the given search methods, we establish a relation between the AI Agent/Search Algorithm, the Graph (Environment), the specific Goal node and the execution of the search strategy to bridge the gap between the start and the goal.

Methodology & Design

Data Structure:

The environment is represented as an List in the code used for this project. Each node contains a list of tuples representing its neighbors and the associated edge costs.

Algorithms Implemented

Breadth-First Search (BFS): Utilizes a First-In-First-Out (FIFO) queue. It explores all nodes at the current depth level before moving to the next.

A Search:* Utilizes a Priority Queue. It selects the next node based on the cost function:

$$f(n) = g(n) + h(n)$$

Where $g(n)$ is the actual cost from the start to node n , and $h(n)$ is the estimated cost (heuristic) from n to the goal.

Heuristic Design

A heuristic system representing the estimated straight-line distance to the goal was used. This ensures that the algorithm never overestimates the cost, guaranteeing an optimal solution.

Implementation (Code)

The implementation was completed in Python, leveraging `collections.deque` for BFS and `heapq` for the A* priority queue to ensure $O(\log n)$ performance for insertions.

```
import collections
import heapq

a = input("Enter start node from A to F")
b = input("Enter finish node from A to F")
```

Results & Comparison

Comparison Table

Metric	BFS	A*
Path Found	A > C > F	A > B > E > F
Path Cost	7	4
Search Nature	Blind/Uninformed	Heuristic/Informed

Observations

Optimality: While BFS found a path with fewer "steps" (2 nodes vs 3 nodes), it failed to find the "cheapest" path because it does not consider edge weights. A* successfully identified the lower-cost route.

Efficiency: A* explored fewer "dead-end" branches (like Node D) because the heuristic value for those nodes was high, lowering their priority in the search queue.

Conclusion

The project successfully demonstrates that adding "intelligence" via a heuristic function significantly optimizes our search functions and their ability to find the best solution. BFS is suitable for simple, unweighted problems, but A* is the better and more reliable method for complex navigation and pathfinding where cost (time, distance, fuel) is a critical factor.

Future Scope

The next iteration of this project will involve:

Integrating NetworkX for dynamic graph generation.

Developing a visual interface to highlight the expansion of nodes in real-time.